

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

CO5 ASSESSMENT TOOL 2 – Seminar on Emerging Technologies

Course Code:	CSA10 / CSA1063	Course Title:	Software Engineering
CO Assessed:	CO5	Assessment:	Assessment Tool 2 – Seminar on Emerging Technologies
Weightage:	20%	Total Marks:	25
CO5 Statement:	Evaluate emerging technologies and societal impacts, maintaining and evolving software systems responsibly		
Student Name:	K.BHAVANA	Reg. No.:	192524178
Date:	20/08/2026	Duration:	60 minutes

Code of Conduct

I K. BHAVANA (192524178) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student:

K. Bhavana

Annexure A – Seminar on Emerging Technologies

Instructions to Students:

Students must select the topic strictly according to the Serial Number (S.No.) order assigned to them.

S.No.	Seminar Topic
1	Generative AI and Large Language Models (LLMs)
2	AI Agents and Autonomous AI Systems
3	Multimodal Artificial Intelligence
4	Explainable and Trustworthy AI
5	Edge AI and Intelligent Edge Computing
6	Federated Learning and Privacy-Preserving AI
7	Deep Learning for Computer Vision
8	AI-Powered Medical Image Analysis
9	Natural Language Processing with Transformer Models
10	Retrieval-Augmented Generation (RAG)
11	AI in Cybersecurity and Threat Detection
12	Zero Trust Architecture and Modern Cybersecurity
13	Post-Quantum Cryptography

14	Quantum Computing and Quantum Algorithms
15	Quantum Machine Learning
16	Cloud-Native Computing and Microservices
17	Serverless Computing and Function-as-a-Service
18	Edge Computing and Fog Computing
19	Cloud Security and Confidential Computing
20	Digital Twins and Intelligent Simulation

21	Internet of Things (IoT) and Smart Systems
22	Industrial Internet of Things (IIoT)
23	5G/6G Networks and Next-Generation Communication
24	Blockchain Technology and Decentralized Applications
25	Web3 and Decentralized Internet
26	Smart Contracts and Blockchain Applications
27	Augmented Reality (AR) and Virtual Reality (VR)
28	Extended Reality (XR) and Immersive Technologies
29	Robotics and Autonomous Robots
30	Human–Robot Interaction and Collaborative Robots
31	Autonomous Vehicles and Intelligent Transportation Systems
32	Smart Cities and AI-Driven Urban Technologies
33	Metaverse Technologies and Virtual Worlds
34	Brain–Computer Interfaces (BCI)
35	Wearable Computing and Smart Wearable Devices
36	Green Computing and Sustainable IT
37	AI for Software Engineering and Code Generation
38	DevSecOps and Intelligent CI/CD Pipelines
39	Digital Forensics and AI-Based Cybercrime Investigation
40	Privacy-Enhancing Technologies and Data Protection

Seminar Presentation Structure:

1. **Introduction to the Emerging Technology**
2. **Need and Motivation**
3. **Working Principle / Architecture**
4. **Key Components and Technologies**
5. **Applications / Real-World Use Cases**
6. **Advantages and Limitations**
7. **Recent Developments and Trends**
8. **Challenges and Future Scope**
9. **Case Study / Example**
10. **Conclusion and References**

BDD-Behavior Driven Development

1. Introduction to the Emerging Technology

Behavior Driven Development (BDD) is an Agile software development methodology that grew out of Test-Driven Development (TDD) to address a very specific, very common failure in software teams: the gap between what the business asks for and what engineers actually build. Rather than treating requirements, tests, and documentation as three separate artifacts maintained by three separate groups, BDD unifies them into a single, plain-language description of system behavior that everyone on the team — business analysts, product owners, testers, and developers — can read, discuss, and agree upon before a single line of production code is written.

The methodology was coined by Dan North in 2003. North, a developer coaching teams in TDD, noticed that developers were often confused about what to test, how much to test, and how to name their tests. He observed that the real question a test should answer is not “does this unit of code work” but “does the system behave the way the business expects it to.” This reframing — from testing implementation details to specifying observable behavior — is the founding insight of BDD, and it is why the methodology is still described today as a natural evolution of TDD rather than a competing practice.

1.1 Defining BDD

At its core, BDD can be summarized by three ideas, often abbreviated as the letters C-B-E:

- **Collaboration** — Business stakeholders, QA engineers, and developers speak one shared language instead of three separate dialects (user stories, test plans, and code).
- **Behavior, not Implementation** — Scenarios describe what the system should do from an outside, user-facing perspective, not how the code internally achieves it.
- **Examples as Specification** — Instead of abstract, ambiguous requirement statements, the team defines requirements using concrete, realistic examples that leave little room for misinterpretation.

Because these examples are written in a structured but still natural-language format, they can be automated and re-run as regression tests. A BDD scenario is therefore simultaneously three things at once: a requirement, a piece of documentation, and an executable test. This triple role is what practitioners mean when they call BDD scenarios “living documentation” — the documentation cannot silently go stale, because if it stops matching the system's real behavior, the automated test built from it fails.

1.2 Where BDD Sits in the Software Development Lifecycle

BDD is not a replacement for Agile or Scrum; it is a practice that fits inside those frameworks, typically during backlog

refinement and sprint planning. Before a user story is considered “ready” for a sprint, the team runs a short collaborative session (see the Three Amigos technique in Section 4) to translate that story into one or more Given-When-Then scenarios. Those scenarios then guide both the developer writing the implementation and the tester validating it, keeping the whole team anchored to the same definition of “done.”

1.3 Historical Background

Before BDD, Dan North was training teams that had adopted TDD but were struggling with three recurring questions: where to start testing, what to test and what to ignore, and how to name a test so its intent was obvious months later. He noticed that teams that answered these questions well were the ones already thinking in terms of behavior — “when the user does X, the system should do Y” — rather than in terms of classes and methods. North formalized this instinct into a naming convention and, soon after, into the Given-When-Then structure that remains the backbone of BDD today. The idea spread quickly through the Extreme Programming and Agile communities and, by the mid-2000s, had produced its first dedicated tooling, JBehave, followed later by the far more widely adopted Cucumber.

1.4 Key Characteristics at a Glance

Characteristic	What It Means in Practice
Ubiquitous language	The same vocabulary is used in conversations, requirements, and code.
Outside-in focus	Design starts from user-visible behavior, not internal architecture.
Executable specification	A written scenario doubles as an automated test.
Collaborative authorship	Scenarios are written with the whole team, not handed down by one role.
Continuous verification	Scenarios re-run on every build, guarding against regressions.

2. Need and Motivation

To understand why BDD exists, it helps to look closely at the problem it was designed to solve: the persistent communication gap between business stakeholders, who know what outcome they need, and developers, who decide how that outcome gets built. Left unmanaged, this gap produces rework, missed deadlines, and software that is technically correct but functionally wrong.

2.1 The Problem: A Widening Communication Gap

- **Ambiguous Requirements** — Business teams typically write user stories in free-form prose. Two developers reading the same story can walk away with two different interpretations, and the mismatch is usually only discovered after the feature is built.
- **Late Bug Discovery** — When requirements are ambiguous, misunderstandings are only caught during QA testing or, worse, after the software has already been released to customers. At that stage, a fix requires re-opening design, code, and test artifacts simultaneously, which is far more expensive than catching the same gap during planning.
- **Tests Written in Isolation** — Traditional unit tests are written by developers, for developers. They are technical, tightly coupled to implementation details, and unreadable to the business stakeholders who are ultimately responsible for signing off on the feature.

2.2 The BDD Solution

- **Shared, Plain Language** — Given-When-Then scenarios are written collaboratively in a structured but human-readable format, so that a business analyst and a senior engineer can read the exact same sentence and agree on what it means.
- **Early Validation (Shift-Left)** — Because examples are discussed and agreed upon before coding starts, disagreements and edge cases surface during planning rather than during testing or production support, which is the essence of the “shift-left” testing philosophy.
- **Executable Specifications** — The same Gherkin scenario that was reviewed and approved by the business is wired to automation code and executed as a regression test — there is no translation step where meaning can be lost.

2.3 BDD in Context: TDD, BDD, and ATDD

BDD is frequently discussed alongside two closely related practices, Test-Driven Development (TDD) and Acceptance Test-Driven Development (ATDD). All three share the philosophy of “test first,” but they differ in scope, audience, and language.

Aspect	TDD (Test-Driven Development)	BDD (Behavior-Driven Development)	ATDD (Acceptance Test-Driven Dev.)
Focus	Code-level correctness	Whole-system, user-facing behavior	Customer-agreed acceptance criteria
Written by	Developers	Whole team (business, QA, dev)	Customer with the team
Language	Programming language (unit tests)	Natural language (Gherkin)	Natural language / criteria lists
Primary audience	Developers	Everyone on the team	Customer and team
Granularity	Class / function level	Feature / scenario level	Acceptance / story level

In short, BDD grew directly out of TDD by shifting the focus from code-level correctness to whole-system behavior, while ATDD is BDD's close sibling, centered specifically on customer-accepted criteria. Many teams use all three together: TDD for unit-level design, BDD for feature-level specification and communication, and ATDD-style acceptance criteria for final customer sign-off.

2.4 The Cost of Getting Requirements Wrong

Industry studies on software rework consistently point to the same conclusion: the earlier a defect or a misunderstanding is caught in the delivery lifecycle, the cheaper it is to fix. A requirement that is misunderstood during planning costs little more than a conversation to correct. The same misunderstanding, if it survives into a released feature, can cost a full development cycle to fix, plus the reputational cost of the bug reaching real customers. BDD's entire value proposition rests on moving the point of discovery for these misunderstandings as far left as possible — ideally into the Three Amigos conversation, before any code exists to be wrong.

2.5 Motivating Question Every Team Should Ask

Dan North summarized the motivation behind BDD in a single question that teams are encouraged to ask before writing any test: “What is the behavior I am trying to specify?” If the answer can be phrased as a concrete Given-When-Then example that a business stakeholder would recognize as correct, the team has a scenario worth automating. If it cannot, that is usually a sign the requirement itself is still too vague to build against — which is precisely the ambiguity BDD exists to surface early.

3. Working Principle / Architecture

BDD does not have a technical “architecture” in the sense of servers or data flows; its architecture is a process architecture — a repeatable structure for turning conversation into code. That structure rests on the Gherkin language and a three-step cycle that moves a scenario from idea to automated regression test.

3.1 The Gherkin Language: Given-When-Then

BDD scenarios are written in Gherkin, a structured, human-readable syntax built around three keywords that map directly onto the anatomy of a behavior:

- **GIVEN** — the initial context or precondition. This sets the scene by describing the state of the system before any action takes place.
- **WHEN** — the action or event that occurs. This describes the specific action that a user or another system performs.
- **THEN** — the expected outcome. This states the observable result that confirms the system behaved correctly.

A minimal example, describing a login feature, illustrates the pattern:

```
Feature: User Login
```

```
  Scenario: Successful login with valid credentials
    Given a registered user is on the login page
    When they enter valid credentials and submit
    Then they should be redirected to their dashboard
```

Notice that this scenario says nothing about databases, session tokens, or which framework renders the login page. It describes only observable behavior from the point of view of the user — which is precisely what makes it readable by a non-technical stakeholder and, at the same time, precise enough to automate.

3.2 The Three-Step Cycle: Discover, Formulate, Automate

Every BDD scenario moves through three stages that together turn a plain-language conversation into a running automated test.

- **1. Discovery** — Business, QA, and developers meet as “Three Amigos” to explore concrete examples of the desired behavior, including edge cases and exceptions, before any scenario is formally written.
- **2. Formulation** — The team turns those examples into structured Given-When-Then scenarios, captured in a feature file that everyone can read and explicitly agree on.
- **3. Automation** — Developers wire the scenario to code using step definitions — small functions that map each Given/When/Then line to executable actions — so the scenario runs automatically as a regression test from that point forward.

This cycle then repeats for the next piece of behavior, so that over the life of a project the feature files accumulate into a comprehensive, continuously verified specification of the whole system.

3.3 From Feature File to Passing Test — the Mechanics

Underneath the plain-language scenario sits an automation layer. A BDD framework (see Section 4.6) parses the feature file and, for every Given/When/Then line, looks up a matching “step definition” written in the team's programming language of choice. That step definition performs the real work — clicking a button through a browser driver, calling an API, or

querying a test database — and asserts that the actual outcome matches the expected outcome in the THEN clause. If it does not, the scenario fails exactly the way a unit test would, but the failure message is phrased in the same business language the team agreed on during discovery, which makes it far easier for a non-programmer to understand what broke.

3.4 Supporting Keywords: And, But, Background, and Scenario Outline

Beyond the core Given-When-Then triad, Gherkin provides a small number of supporting keywords that keep larger feature files readable and avoid repetition.

- **And / But** — used to chain multiple conditions or outcomes under a single Given, When, or Then without repeating the keyword, as seen in the checkout and ATM examples in Section 5.
- **Background** — a block of Given steps that apply to every scenario in a feature file, avoiding the need to repeat the same setup steps in each individual scenario.
- **Scenario Outline / Examples** — used when the same behavior needs to be verified against multiple sets of input data, letting one scenario template stand in for many concrete test cases.

The Scenario Outline pattern is especially useful for data-driven testing. The ATM withdrawal rule from Section 5.2, for instance, can be generalized to cover several balance-and-request combinations at once:

```
Feature: ATM Cash Withdrawal
```

```
Scenario Outline: Withdrawal request compared to balance
```

```
Given the account balance is ₹<balance>
```

```
When the customer requests to withdraw ₹<amount>
```

```
Then the transaction should be "<result>"
```

```
Examples:
```

balance	amount	result
500	1000	rejected
500	300	approved
500	500	approved

A single Scenario Outline like this expands, at run time, into three independent test executions — one per row of the Examples table — giving thorough edge-case coverage without duplicating the scenario text three times over.

4. Key Components and Technologies

BDD is supported by a small set of named techniques that structure the collaboration, plus a family of tools that turn Gherkin scenarios into automated tests. Together they form the practical toolkit a team needs to adopt BDD.

4.1 Three Amigos

The “Three Amigos” meeting brings together a business representative, a developer, and a tester (QA) before coding begins. Each perspective surfaces different concerns — the business representative worries about value and correctness, the developer worries about feasibility and edge cases, and the tester worries about what could go wrong — and the resulting scenarios are stronger for having been challenged from all three angles.

4.2 Specification by Example

Rather than describing requirements abstractly (“the system should validate payments correctly”), the team illustrates the requirement with concrete, realistic examples (“given a cart total of ₹1,999 and a card limit of ₹2,000, the payment should be accepted”). Concrete examples are far less ambiguous than abstract descriptions and are easier for all parties to validate.

4.3 Gherkin / Given-When-Then

Covered in detail in Section 3.1, Gherkin is the structured natural-language syntax that makes scenarios both human-readable and machine-parseable.

4.4 Living Documentation

Because feature files are stored alongside the code and run as part of the automated test suite, they cannot silently drift out of date the way a static requirements document can. If a scenario no longer matches the deployed behavior, either the scenario is updated deliberately (reflecting a conscious change in requirements) or the test fails, immediately signalling a regression.

4.5 Outside-In Development

BDD encourages designing a feature from the outside in: start from the user-facing behavior described in a scenario, and only then work inward toward the classes, functions, and data structures needed to satisfy it. This keeps implementation decisions anchored to real user value rather than to what is technically convenient to build first.

4.6 Automated Acceptance Testing and Tooling

Scenarios are wired to automation code using a BDD framework. The most widely used tools include:

Tool	Primary Language(s)	Notes
Cucumber	Java, Ruby, JavaScript	The original and most widely adopted Gherkin-based BDD framework; large plugin ecosystem.
SpecFlow	.NET (C#)	Cucumber-equivalent for the .NET ecosystem, integrates tightly with Visual Studio.
Behave	Python	Lightweight Gherkin runner popular in Python test automation stacks.
JBehave	Java	One of the earliest BDD frameworks, predates Cucumber.

Tool	Primary Language(s)	Notes
Jest-Cucumber / CucumberJS	JavaScript / TypeScript	Used for BDD in Node.js and front-end test suites.

Regardless of the specific tool, the underlying workflow is the same: a Gherkin feature file is parsed, each step line is matched to a step definition function, and the framework's runner reports which scenarios passed and which failed, in a format that both a developer and a business stakeholder can read.

5. Applications and Real-World Use Cases

BDD is applied wherever a team needs high confidence that software behaves correctly from a user's or business's point of view, and where the cost of a misunderstood requirement is high. Two detailed, worked examples illustrate the practice, followed by a broader survey of industries where BDD has become common.

5.1 Worked Example: E-Commerce Online Checkout

Consider a shopping website. Before developers write any code, the product team, QA, and developers agree on the following scenario together in a Three Amigos session:

Feature: Shopping Cart Checkout

Scenario: Successful checkout with valid payment

Given a customer has 2 items in their cart

And the total is within their card limit

When the customer submits valid payment details

Then the order is confirmed

And a receipt is emailed to the customer

This same block of text is then automated with a tool such as Cucumber, so that it runs as a regression test on every future code change. If the checkout logic is ever broken by a later change — for example, a refactor that accidentally skips sending the receipt email — this scenario fails immediately during the build pipeline, catching the defect before it ever reaches a paying customer.

5.2 Worked Example: Banking — ATM Cash Withdrawal

A second example, taken from a safety-critical domain, shows how BDD handles negative and edge cases as rigorously as it handles the “happy path”:

Feature: ATM Cash Withdrawal

Scenario: Withdrawal exceeds available balance

Given the account balance is ₹500

When the customer requests to withdraw ₹1000

Then the ATM should reject the transaction

And display "Insufficient funds"

And return the customer's card

Banking rules — such as rejecting an overdraft request — are exactly the kind of edge case that business analysts and

compliance officers must verify before release. Writing the rule in plain English lets a non-programmer confirm that the requirement has been correctly captured and implemented, long before the feature reaches a formal audit or a customer-facing incident.

5.3 Worked Example: Healthcare — Patient Appointment Scheduling

BDD is equally applicable outside e-commerce and banking. Consider a hospital's appointment-booking system, where a regulatory requirement states that a patient must never be double-booked into two overlapping slots with the same doctor:

Feature: Patient Appointment Scheduling

Scenario: Prevent double-booking for the same doctor

Given Dr. Rao has an appointment booked from 10:00 to 10:30

When a receptionist tries to book another patient with Dr. Rao at 10:15

Then the system should reject the booking

And display "Time slot unavailable"

As with the banking example, writing this rule in Gherkin lets a hospital administrator — who is not a programmer — confirm that a patient-safety rule has been captured correctly, and lets an auditor later verify the rule is still enforced by re-running the same scenario against the live system.

5.4 Where BDD Is Used Across Industries

Domain	Typical Use of BDD
Agile & Scrum Teams	Clarifying user stories during sprint planning and backlog refinement.
E-Commerce Platforms	Validating cart, checkout, pricing, and promotion logic end-to-end.
Banking & Fintech	Verifying compliance-critical rules such as transaction limits and fraud checks.
Healthcare Systems	Ensuring patient-data workflows meet strict regulatory requirements.
API & Microservices Testing	Describing service contracts and integration behavior in readable scenarios.
Regression Test Suites	Building a growing, reusable library of automated acceptance tests.

6. Advantages and Limitations

6.1 Advantages of BDD

- **Improved Communication** — Creates a shared, unambiguous language between business and technical teams, reducing the number of “that's not what I meant” conversations.
- **Single Source of Truth** — Requirements, tests, and documentation live in one synchronized artifact instead of three separate documents that can drift apart.
- **Early Defect Detection** — Misunderstandings surface during specification rather than after release, in line with the shift-left testing philosophy.
- **Living Documentation** — Feature files always reflect the current, true behavior of the system, unlike static Word or PDF specification documents that go stale the moment the code changes.
- **Reusable Regression Suite** — Automated scenarios accumulate into a growing safety net that protects against future regressions as the codebase grows in size and complexity.
- **Stakeholder Confidence** — Non-technical stakeholders can read the tests themselves and verify that the system does what they asked for, without needing a developer to translate.

6.2 Disadvantages and Challenges of BDD

- **Time and Effort Overhead** — Writing and maintaining detailed scenarios takes significant upfront time, particularly for large or fast-changing systems.
- **Requires Cultural Buy-In** — BDD fails if business stakeholders are unwilling, too busy, or unavailable to collaborate in Three Amigos sessions; without their participation, scenarios revert to being developer-only artifacts.
- **Feature File Bloat** — Overuse can produce hundreds of overlapping, poorly organized scenarios that become as hard to maintain as the code they test.
- **Learning Curve** — Teams must learn Gherkin syntax and new tooling such as Cucumber or SpecFlow, which takes time away from feature delivery in the short term.
- **Not a Full Test Replacement** — BDD covers acceptance-level behavior; it does not replace detailed unit and integration testing, which are still needed to catch lower-level defects.
- **Overkill for Small Projects** — For simple, short-lived applications, the process overhead of Three Amigos sessions and Gherkin authoring may not be justified by the benefit gained.

Taken together, these advantages and challenges suggest that BDD delivers the most value on collaborative, complex, or compliance-heavy projects with a reasonably long lifespan, and comparatively less value on small, throwaway, or purely internal tooling projects where the communication gap it solves barely exists.

7. Recent Developments and Trends

Since Dan North's original formulation in 2003, BDD has matured from a niche testing technique into a mainstream Agile practice, and it continues to evolve alongside the broader software delivery ecosystem.

7.1 Integration with CI/CD Pipelines

Modern teams no longer run BDD scenarios only on demand; feature files are wired directly into continuous integration and continuous delivery (CI/CD) pipelines, so that every commit automatically triggers the full acceptance-test suite. A failing scenario can block a merge or a deployment outright, turning living documentation into an active quality gate rather than a passive record.

7.2 Cloud-Based and Cross-Browser Test Execution

Cloud testing platforms now allow Gherkin-driven acceptance tests to run in parallel across many browser, device, and operating-system combinations simultaneously, dramatically shortening feedback cycles for teams that previously ran such suites sequentially on local machines.

7.3 AI-Assisted Scenario Authoring

A newer trend is the use of AI coding assistants to help draft Gherkin scenarios and their corresponding step definitions from a plain description of a feature, and to suggest missing edge cases during Three Amigos-style discussions. This lowers the authoring overhead that has historically been one of BDD's most cited drawbacks, though human review remains essential to keep the language genuinely shared and unambiguous rather than machine-generated boilerplate.

7.4 BDD Beyond the Web: APIs, Microservices, and IoT

As architectures have shifted toward microservices, BDD has followed: teams now write Given-When-Then scenarios describing service contracts and inter-service behavior, not just browser-based user journeys. The same approach is increasingly applied to Internet-of-Things systems, where a scenario might describe how a sensor reading should trigger a downstream action.

7.5 Low-Code and No-Code BDD Authoring Tools

A growing category of tools lets business stakeholders write and maintain Gherkin scenarios directly through a web interface, with the underlying automation code generated or mapped separately by engineers. This further narrows the gap BDD was originally designed to close, letting non-technical users own more of the specification process end to end.

8. Challenges and Future Scope

8.1 Persistent Challenges

- **Scenario Maintenance at Scale** — As a product grows, the sheer number of feature files can become difficult to organize, deduplicate, and keep consistent, requiring disciplined naming conventions and periodic pruning.
- **Balancing Detail and Readability** — Scenarios that are too detailed become brittle and hard to read; scenarios that are too abstract lose their value as precise specifications. Finding the right level of granularity remains a skill that takes teams time to develop.
- **Sustaining Business Engagement** — Even teams that adopt BDD successfully at first can see business participation fade over time as deadlines tighten, causing scenarios to gradually become developer-only artifacts again.
- **Tooling Fragmentation** — Different frameworks (Cucumber, SpecFlow, Behave, and others) have different plugin ecosystems and IDE support, which can complicate standardization across a large organization with multiple technology stacks.

8.2 Future Scope

- **Deeper AI Integration** — As generative AI tools mature, expect closer integration between plain-language feature requests and automatically generated, human-reviewed Gherkin scenarios and step definitions, further lowering the barrier to entry.
- **Unified Specification Platforms** — Emerging tools aim to unify requirement management, Gherkin authoring, and test execution reporting into a single collaborative platform, reducing the current fragmentation across separate tools.
- **Wider Adoption in Regulated Industries** — As compliance requirements grow more complex in domains like healthcare, banking, and critical infrastructure, BDD's ability to produce auditable, plain-language, executable specifications positions it for continued growth in exactly these safety-critical settings.
- **Extension to Non-Functional Behavior** — There is growing interest in extending the Given-When-Then pattern beyond functional correctness to describe performance, security, and accessibility expectations in the same shared language.

8.3 Challenges Mapped to Mitigations

Challenge	Practical Mitigation
Scenario maintenance at scale	Adopt consistent tagging and folder conventions; periodically review and retire obsolete scenarios.
Balancing detail and readability	Keep scenarios at the business-behavior level; push technical detail into step definitions, not Gherkin text.
Sustaining business engagement	Timebox Three Amigos sessions and keep them focused on new or risky behavior, not every minor change.
Tooling fragmentation	Standardize on one BDD framework per technology stack and document conventions centrally.

9. Case Study / Example

9.1 Case Study: Adopting BDD for an Online Retail Platform

To see how the individual techniques discussed above combine in practice, consider a composite, illustrative case study modeled on the e-commerce checkout example introduced in Section 5.1. An online retail platform's checkout flow had a history of subtle bugs — orders occasionally confirmed without sending a receipt, and payment validation logic disagreed with what the finance team believed the rules were. The team decided to introduce BDD specifically for the checkout module.

Step 1 — Discovery

The product owner, a QA engineer, and a backend developer held a Three Amigos session focused solely on checkout. They walked through the happy path (successful payment), several edge cases (payment declined, cart total at exactly the card limit, and an item going out of stock mid-checkout), and one compliance requirement (a receipt must always be emailed, without exception, for audit purposes).

Step 2 — Formulation

The team converted the happy-path discussion into the Gherkin scenario shown earlier in Section 5.1, and wrote sibling scenarios for each edge case identified during discovery, including an explicit scenario asserting that a receipt is emailed even when the confirmation page fails to load — directly addressing the compliance requirement.

Step 3 — Automation

Developers implemented step definitions using Cucumber, connecting each Given/When/Then line to calls against a staging checkout API. The scenarios were added to the CI/CD pipeline so that every pull request touching the checkout module automatically re-ran the full set.

Outcome

Within the following release cycle, the automated checkout scenarios caught a regression where a promotional-discount change accidentally skipped the receipt email for orders paid with a stored card. Because the scenario describing that exact rule already existed and ran on every commit, the defect was caught in the pull request itself rather than after release, and the responsible engineer could immediately see, in plain language, which business rule had been broken.

9.2 Lesson from the Case Study

The case study illustrates the central claim made throughout this report: BDD's value comes not from Gherkin syntax alone, but from the discipline of discovering edge cases collaboratively before coding, and from keeping the resulting scenarios alive as automated regression tests. A team that skips the Three Amigos discovery step and writes Gherkin scenarios alone, after the fact, captures the syntax of BDD without capturing the communication benefit that makes it worthwhile.

9.3 Before-and-After Snapshot

Metric	Before BDD Adoption	After BDD Adoption
Checkout-related production defects / quarter	Recurring, discovered post-release	Caught in pull request via failing scenario
Requirement ownership	Prose user stories, single interpretation risk	Jointly agreed Given-When-Then scenarios
Regression coverage for checkout	Manual QA pass before each	Automated scenario suite run on

Metric	Before BDD Adoption	After BDD Adoption
	release	every commit
Time to detect a broken business rule	Days (post-release monitoring)	Minutes (CI pipeline failure)

10. Conclusion and References

10.1 Conclusion

- **Reframes the core question** — BDD shifts focus from “how do we build it” to “what behavior do we need,” keeping business value central to every technical decision.
- **Provides a shared contract** — Plain-language, Given-When-Then scenarios act as a contract that business and engineering both understand and can hold each other to.
- **Doubles as documentation and test** — The same scenario acts as an automated acceptance test and as always-current living documentation, reducing the drift between what is built and what is written down.
- **Fits best where it is needed most** — BDD is best suited to collaborative, complex, or compliance-heavy projects; for tiny or short-lived applications, its process overhead may outweigh the benefit.

In summary, Behavior Driven Development is less a testing tool than a communication discipline that happens to produce automated tests as a byproduct. Its lasting contribution to software engineering is the insight that the most expensive bugs are not coding mistakes but misunderstandings — and that the cheapest way to fix a misunderstanding is to write it down, in plain language, before any code exists to misunderstand.

10.2 References

1. North, D. (2006). “Introducing BDD.” Dan North & Associates.
2. Wynne, M., Hellesoy, A., & Tooke, S. (2017). The Cucumber Book: Behaviour-Driven Development for Testers and Developers. Pragmatic Bookshelf.
3. Adzic, G. (2011). Specification by Example: How Successful Teams Deliver the Right Software. Manning Publications.
4. Cucumber Limited. “Cucumber Documentation.” cucumber.io/docs — official Gherkin and Cucumber reference.
5. SpecFlow / Tricentis. “SpecFlow Documentation.” specflow.org/docs — .NET BDD framework reference.
6. Bhavana, K. (2026). “Behavior Driven Development (BDD).” Seminar presentation, CSA1063 Software Engineering, SIMATS Engineering.
7. SIMATS Engineering, Department of CSE. “CO5 Assessment Tool 2 — Seminar on Emerging Technologies” (course assessment brief), CSA10 / CSA1063 Software Engineering.

Appendix A — Sample Automation Code (Cucumber / Java)

To make the automation layer described in Section 3.3 concrete, this appendix shows how the e-commerce checkout scenario from Section 5.1 is wired to executable step definitions using Cucumber for Java. The feature file is the same plain-language scenario the business already reviewed and approved; only the step definitions below are new, technical, and owned by developers.

A.1 The Feature File

Feature: Shopping Cart Checkout

Scenario: Successful checkout with valid payment
Given a customer has 2 items in their cart
And the total is within their card limit
When the customer submits valid payment details
Then the order is confirmed
And a receipt is emailed to the customer

A.2 The Step Definitions (Java)

```
public class CheckoutSteps {

    private Cart cart = new Cart();
    private OrderResult result;

    @Given("a customer has {int} items in their cart")
    public void addItemToCart(int itemCount) {
        cart.addSampleItems(itemCount);
    }

    @Given("the total is within their card limit")
    public void totalWithinLimit() {
        assertTrue(cart.getTotal() <= paymentCard.getLimit());
    }

    @When("the customer submits valid payment details")
    public void submitPayment() {
        result = checkoutService.submit(cart, paymentCard);
    }

    @Then("the order is confirmed")
    public void orderIsConfirmed() {
        assertEquals(OrderStatus.CONFIRMED, result.getStatus());
    }

    @Then("a receipt is emailed to the customer")
    public void receiptIsEmailed() {
        assertTrue(mailService.wasReceiptSentFor(result.getOrderId()));
    }
}
```

```
}  
}
```

Each annotated method corresponds to exactly one line of the feature file. When the Cucumber runner executes the scenario, it reads the Gherkin text, matches each line to its annotated method by pattern, runs the methods in order, and reports the scenario as passed only if every assertion succeeds — producing a result that a business stakeholder can interpret without reading a single line of the Java code itself.

Appendix B — Glossary of Key Terms

Term	Definition
Feature File	A plain-text file, usually with a .feature extension, containing one or more Gherkin scenarios describing a single feature.
Scenario	A single, concrete example of system behavior, expressed as a Given-When-Then sequence.
Step Definition	A snippet of code that maps a single Given, When, or Then line to an executable action and assertion.
Gherkin	The structured, keyword-based natural language used to write BDD scenarios.
Living Documentation	Documentation (here, feature files) that is automatically kept accurate because it doubles as an executable test suite.
Three Amigos	A collaborative meeting of a business representative, a developer, and a tester held before coding begins.
Shift-Left Testing	The practice of moving testing and validation activities earlier in the development lifecycle.
Regression Test	A test that re-verifies previously working behavior has not broken as a result of a later code change.
Scenario Outline	A Gherkin template scenario that runs once for each row of an accompanying Examples data table.
Acceptance Criteria	The conditions a piece of software must satisfy to be accepted by the customer or business owner.

Behavior Driven Development (BDD)

Bridging the gap between business goals and technical implementation through a shared, plain-language understanding of software behavior.

Presented by:
K. Bhavana (192524178)

GPS Map Camera

Mevalurkuppam, Tamil Nadu, India



22g8+9hp, Mevalurkuppam, Tamil Nadu 602105, India

Lat 13.025896° Long 80.016576°

Thursday, 20/08/2026 08:33 AM GMT +05:30

Google

Rubrics for Calculation

Evaluation Criteria	Excellent (4 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1 Mark)
Technical Content & Subject	Comprehensive and accurate content; demonstrates excellent	Good technical understanding with minor gaps	Basic understanding with limited technical depth	Incomplete content or major technical errors

Knowledge (25%)	understanding of the emerging technology			
Organization & Presentation Structure (15%)	Excellent logical flow with clear introduction, content, conclusion, and effective transitions	Well-structured with minor organizational issues	Adequately structured but some sections lack clarity	Poorly structured and difficult to follow
Examples, Diagrams & Case Study (20%)	Excellent use of relevant diagrams, examples, applications, and real-world case studies	Good use of relevant examples and diagrams	Limited examples/diagrams with basic explanation	Inappropriate or insufficient examples/diagrams
Communication & Presentation Skills (20%)	Very clear, confident, engaging delivery with excellent eye contact, voice modulation, and time management	Clear and confident presentation with minor issues	Understandable but lacks confidence or engagement	Unclear delivery, excessive reading, or poor time management
Question Handling & Overall Performance (20%)	Answers questions accurately and confidently; demonstrates strong overall knowledge	Answers most questions correctly	Answers basic questions with some difficulty	Unable to answer questions or demonstrate adequate understanding

Criteria	Mark
Technical Content & Subject Knowledge (25%)	/5
Organization & Presentation Structure (15%)	/5
Examples, Diagrams & Case Study (20%)	/5
Communication & Presentation Skills (20%)	/5
Question Handling & Overall Performance (20%)	/5
TOTAL	/25